

An Exact Snoop Filter for Seamless Cache Coherence in FPGA-Based Multi-Core RISC-V

Anonymous submission

Abstract—Private data caches allow the soft cores of an FPGA-based multi-core system to hide the access latency of a shared memory, but the caches then have to be kept coherent. A recently proposed cache extension for RISC-V multi-core systems does this in hardware, with a snooping invalidation bus and without any modification to the core. In this paper, the traffic on that bus is examined. The cache extension is reproduced at the register-transfer level, its snoopy bus is instrumented, and the number of invalidation requests that actually invalidate a cache line in another core is counted. On the benchmark kernels of the original evaluation this number is zero: the kernels partition their data between the processing elements, so every written line is private to the core that writes it, while each write is still arbitrated across the whole system and acknowledged by every other cache. A considerable fraction of each core’s execution time is spent in this way, and for a streaming kernel the overhead outweighs the benefit of the cache. We propose a snoop filter that keeps a copy of the tag and valid bits of every data cache unit inside the snoopy bus, so that the bus can suppress an invalidation request when no other cache holds the addressed line or is fetching it. A cost model of the bus protocol explains where the saving comes from, and the filter is evaluated in execution time, coherence correctness and FPGA resource utilization.

Index Terms—Cache Coherency, Snoop Filter, RISC-V, Multi-Core SoC, FPGA

I. INTRODUCTION

Multi-core systems implemented on FPGAs are increasingly used for embedded and edge computing, where several soft RISC-V cores share a common memory whose access latency is high compared to the clock period of the cores. Private data caches hide this latency, but as soon as each core caches shared data the copies have to be kept coherent, and the cost of coherence is charged against the same performance the caches were added to provide: as hardware for tag and status memories, and as the time the cores spend waiting for coherence actions that are serialized across the whole system.

Several hardware and software coherence schemes have been proposed for scalable architectures [2]–[4], and the RISC-V ecosystem provides coherent interconnect fabrics for large systems [5]–[7]. Adopting one of these means adopting its interconnect and, in most cases, modifying the core, which is exactly the development cost a designer working with existing soft cores wants to avoid. A seamless cache extension avoids it. Kamaleldin *et al.* [1] attach a private, write-through Data Cache Unit (DCU) to an unmodified RISC-V core through the core’s ordinary data port and keep the DCUs coherent with a snooping invalidation bus, completely in hardware.

In this work, we reproduce the cache extension of [1] at the register-transfer level in order to measure how much of the coherence traffic it generates is actually necessary. We find

that between 6% and 33% of the time a core spends in its cache goes to waiting for the snoopy bus, and that across four benchmark kernels the system issues 1,048,821 invalidation requests of which not one invalidates a cache line in another DCU. The kernels partition their arrays between the Processing Elements (PEs), so almost every written line is private to one PE, but the bus has no way of knowing this. We therefore propose an exact snoop filter that answers this question before the bus arbitrates the request. Our contributions are listed as follows:

- A measurement of how much of the coherence traffic of the cache extension is necessary and what it costs (Section IV).
- An exact snoop filter, a structural mirror of the tag and valid bits of all DCUs that also covers lines being fetched, and a cost model of the bus protocol that derives its benefit (Section V).
- An evaluation in execution time, coherence correctness and FPGA resources, and three rejected alternatives (Sections VI and VII).

The rest of the paper is structured as follows. Section II discusses related work, Section III describes the cache extension under study, and Section IV analyzes its coherence traffic. The snoop filter is presented in Section V and evaluated in Section VI; Sections VII to IX report the rejected mechanisms, the limitations of this study, and the conclusion.

II. RELATED WORK

Snoop filtering is an established idea; the contribution here is the position of the filter, the property it is exact about, and the cost it is measured against. JETTY [2] places a small cache-like structure between the bus and the second-level cache of each processor and filters the snoops that would miss locally, so that the energy of a tag lookup is not spent on them. It sits on the receiving side of the bus and is approximate. Coarse-grain coherence tracking [3] and RegionScout [4] instead track the sharing state of large regions of memory, so that a region known to be unshared needs no broadcast at all.

In a bus-based soft multi-core system the cost that matters is different. The expense of an unnecessary invalidation is not the energy of the remote tag lookups but the serialization it imposes: only one invalidation is granted per clock cycle in the whole system, and the grant is held back until every other cache is ready to accept it. The filter therefore belongs at the source of the request, where it can decide whether to arbitrate at all, and since the caches are small, an exact copy

of their tag arrays is affordable. Coherent RISC-V fabrics such as TileLink [5], OpenPiton [6] and CIFER [7] are complete memory systems; the design studied here is an extension attached to an unmodified core through its existing memory port, which confines its coherence traffic to a single shared bus.

III. THE CACHE EXTENSION UNDER STUDY

This section describes the cache extension of [1] to the extent needed to follow the measurements, using the terminology of the original paper.

A. Multi-core RISC-V Architecture

The target system, shown in Fig. 1, consists of a scalable number of RISC-V Processing Elements (PEs). Each PE hosts an in-order RV32 core, a private Data Cache Unit (DCU), a scratchpad Instruction Tightly Coupled Memory (ITCM), and an Instruction-Bridge and a Data-Bridge that connect the core to the local memories and to a shared AXI4 crossbar, behind which sit a shared instruction memory, used as boot memory, and a coherent shared data memory. A single snoopy bus is connected to the DCU of every PE and manages the invalidation of cache lines between the private DCUs.

B. Data Cache Unit

The DCU is a two-way set-associative, write-through, no-allocate data cache with a Least Recently Used (LRU) replacement policy: every write is forwarded to the shared data memory, which therefore always holds the current value of every location, and a write that misses does not install a line. The DCU processes requests in two sequential pipeline stages. The first stage accepts a request either from the local core, which can be a read request (READ REQ) or a write request (WRITE REQ), or from the snoopy bus, which can only be an invalidation request (INV REQ). For a core request, the first stage reads the valid bit from the Status-RAM and the tag from the Tag-RAM and at the same time forwards the request to the snoopy bus, a WRITE REQ as an INV REQ. In the second stage, the Hit Check Logic (HCL) decides between hit and miss and the Cache Control Logic (CCL) acts on the request: a READ REQ that misses issues a memory read request (MEM READ REQ) and fills the LRU way of the set with the returned line, a WRITE REQ is forwarded as a memory write request (MEM WRITE REQ), and an INV REQ clears the valid bit of the addressed line if it is resident. An INV REQ has priority over the local core: if it arrives while a core request occupies the first stage, that request is displaced into a holding register and re-presented once the INV REQ has moved on. A Stall Control Logic (SCL) stalls the first stage until the bus grants its forwarded request, and the second stage until the response (MEM RESP) of a MEM READ REQ or the grant of a MEM WRITE REQ arrives, i.e. for the access latency of the shared data memory.

C. Snoopy Bus

The snoopy bus consists of two Round Robin (RR) arbiters, an Invalidation Table and a Link Register. The first arbiter, called the INV REQ arbiter in the following, arbitrates the INV REQs of all PEs and grants at most one per clock cycle. When an INV REQ is granted, its address is broadcast to the DCUs of all other PEs, which clear the addressed line if they hold it; the grant is withheld until every other DCU is able to accept the broadcast, so that no invalidation is lost. The second arbiter handles the atomic Load-Reserved (LR) and Store-Conditional (SC) requests with the help of the Link Register. The bus guarantees a Multiple-Readers-Single-Writer (MRSW) invariant: plain READ REQs are not arbitrated, but while a core has an INV REQ pending for a line, recorded in the Invalidation Table, a READ REQ of another core to the same line is withheld until the INV REQ has been granted.

D. Reproduction and Experimental Setup

Our reproduction implements this architecture in SystemVerilog in the configuration stated by the original paper: four PEs, each with an unmodified CV32E40P core and a private 2 KiB two-way DCU with 16-byte lines and 64 sets, a 256 KiB shared data memory and a 32 KiB shared instruction memory behind an AXI4 crossbar. A non-coherent version, in which each core accesses the shared data memory directly through its Data-Bridge without a DCU, is built from the same sources by changing one parameter and is the baseline for all comparisons. The shared memories are modelled with a configurable access latency, called the memory latency in the following: the number of clock cycles between the acceptance of a MEM READ REQ and the first data beat of its MEM RESP. Two cycles corresponds to on-chip block RAM (BRAM), as in the original design; eight and twenty cycles represent off-chip memories.

Execution time is measured with the cycle counter of the cores as the number of clock cycles until the slowest PE has finished. The benchmark kernels are the memory copy, matrix multiplication, 2-D convolution and Fast Fourier Transform (FFT) kernels of the original evaluation, referred to as memcpy, matmul, conv2d and FFT; each partitions its data between the four PEs, and every run is checked against a golden result. Each DCU also carries hardware counters for the cycles it holds a core request (busy cycles), the cycles its first stage waits for a grant of the snoopy bus (snoop-stall cycles), the INV REQs it receives, and those among them that find a valid matching line to clear.

IV. ANALYSIS OF THE COHERENCE TRAFFIC

A. Time Spent Waiting for the Snoopy Bus

We express the snoop-stall cycles of a DCU as a fraction of its busy cycles, i.e. the share of a core's cache time that goes to waiting for the bus. At a memory latency of two cycles this share is 14.3 % on memcpy, 15.8 % on matmul, 6.5 % on conv2d and 5.8 % on FFT; at a memory latency of twenty it rises to 33.0 %, 32.9 %, 19.0 % and 12.2 %, respectively. The share grows with the memory latency because, as shown

mirror learns the new tag, so the line is covered in every cycle by one of the two. This is not merely a defensive measure: over ten randomized interleavings of our coherence testbench, 20 out of 2,241 granted INV REQs addressed a line that no other DCU held but one DCU was fetching, i.e. a broadcast that a mirror of resident lines alone would have suppressed wrongly.

C. Correctness of Suppression

When the filter reports that no other DCU holds the addressed line or is fetching it, a broadcast of the INV REQ could not clear a line anywhere, and not sending it is unobservable. This is the whole correctness argument. Everything else in the bus is unchanged: INV REQs that are needed are still arbitrated, broadcast and acknowledged as before, and the Invalidation Table still withholds READ REQs according to the MRSW invariant.

D. Cost Model

Here we derive from the bus protocol what suppressing a broadcast is worth, and check the derivation against the measurements. Let P be the number of PEs, indexed by i and j . For a WRITE REQ w issued by PE i , let $a(w)$ be the clock cycle in which w enters the first stage of its DCU and $g(w)$ the cycle in which the corresponding INV REQ is granted; let $\text{arb}(t)$ be the PE selected by the INV REQ arbiter in cycle t , and $r_j(t) = 1$ if the DCU of PE j can accept an INV REQ in cycle t . The sharer set $S(w)$ is the set of PEs other than i whose DCU holds or is fetching the line addressed by w . The original protocol grants an INV REQ in the first cycle in which the arbiter selects it *and* every other DCU is ready to accept it:

$$g(w) = \min \left\{ t \geq a(w) : \text{arb}(t) = i \wedge \bigwedge_{j \neq i} r_j(t) \right\}. \quad (1)$$

A DCU is ready only while its first stage does not already hold an INV REQ, and an INV REQ leaves the first stage only when the second stage is free, which it is not for the full memory latency of any MEM READ REQ or MEM WRITE REQ in progress. On grant, one INV REQ is inserted into the first stage of each of the other $P-1$ DCUs, where it has priority over the local core, so the pipeline of each of them is delayed by one cycle if it had a core request to process. The cost of w to the whole system is therefore

$$C(w) = (g(w) - a(w)) + \sum_{j \neq i} \rho_j(g(w)), \quad (2)$$

where $\rho_j(t) \in \{0, 1\}$ is one if the DCU of PE j had a core request to delay in cycle t . The conjunction in (1) couples the pipelines of all PEs: a WRITE REQ on PE i waits until a MEM READ REQ on PE j has completed, whether or not the two PEs share any data, which is why the snoop-stall share of Section IV grows with the memory latency. Neither term of (2) depends on which lines the caches hold.

The filter makes the cost depend on the cache state. With $S(w)$ read from the mirror, the grant condition becomes $\text{arb}(t) = i \wedge (S(w) = \emptyset \vee \bigwedge_{j \neq i} r_j(t))$, and INV REQs are

TABLE I
EXECUTION TIME IN CLOCK CYCLES OF THE SLOWEST PE, FOR THE VALUES PLOTTED IN FIG. 2. THE LAST TWO COLUMNS GIVE THE REDUCTION RELATIVE TO THE NON-COHERENT SYSTEM WITHOUT A DATA CACHE, FOR THE CACHE EXTENSION ALONE AND WITH THE PROPOSED FILTER ADDED.

Kernel	Size	Execution cycles			Reduction	
		No cache	Cache	+Filter	Cache	+Filter
matmul	32^2	277,621	171,142	140,223	38.4%	49.5%
	64^2	2,191,981	1,337,850	1,094,750	39.0%	50.1%
	128^2	16,899,384	9,461,615	8,118,893	44.0%	52.0%
conv2d	32^2	73,268	38,120	35,526	48.0%	51.5%
	64^2	311,454	168,177	154,382	46.0%	50.4%
	128^2	1,277,966	683,079	625,110	46.5%	51.1%
FFT	256	113,092	85,093	80,228	24.8%	29.1%
	512	245,292	244,338	225,852	0.4%	7.9%
	1024	550,489	574,366	541,289	-4.3%	1.7%
FFT, blocked	256	121,213	93,087	88,604	23.2%	26.9%
	512	269,224	238,776	219,017	11.3%	18.6%
	1024	584,916	523,833	478,210	10.4%	18.2%

inserted into the other DCUs only when $S(w) \neq \emptyset$. For a WRITE REQ with an empty sharer set, both terms of (2) vanish and only the arbitration wait remains, which is non-zero only when two PEs present a WRITE REQ in the same cycle. Summed over a kernel in which every WRITE REQ has an empty sharer set, the case Section IV found for all four kernels, the saving is the whole coherence overhead minus the arbitration collisions, which is exactly the saving of an oracle that suppresses every broadcast.

Both terms of (2) are observable, so the model can be checked rather than fitted: the first term is the snoop-stall counter, and the second is at most one cycle per INV REQ delivered. The snoop-stall cycles removed by the filter are therefore a lower bound on the saving, and the same plus one cycle per broadcast received an upper bound. Per DCU, at a memory latency of two, the measured saving lies inside this bracket on every kernel: matmul $120,539 \leq 243,100 \leq 326,373$ cycles, conv2d $5,185 \leq 13,795 \leq 39,980$, FFT $892 \leq 4,865 \leq 16,312$, memcpy $1,792 \leq 2,302 \leq 7,946$.

In hardware, the mirror is one memory array per (PE, way) pair, addressed by the set index. The tags are held outside any reset, since a tag is only compared when its valid bit is set, which lets them map to distributed RAM (LUTRAM); the valid bits are held in flip-flops, where they can be reset and read for all arrays at once. A first implementation that held both under an asynchronous reset was mapped by Vivado entirely to flip-flops, 11,776 of them and 3,202 LUTs, compared to 512 and 408 for this structure.

VI. EVALUATION AND PERFORMANCE ANALYSIS

A. Execution Time

Fig. 2 shows the execution time of the matmul, conv2d and FFT kernels for the problem sizes of the original evaluation, and Table I lists the cycle counts behind it. For matmul and conv2d, the cache extension reduces the execution time by approximately 40 % at every problem size, as in the original

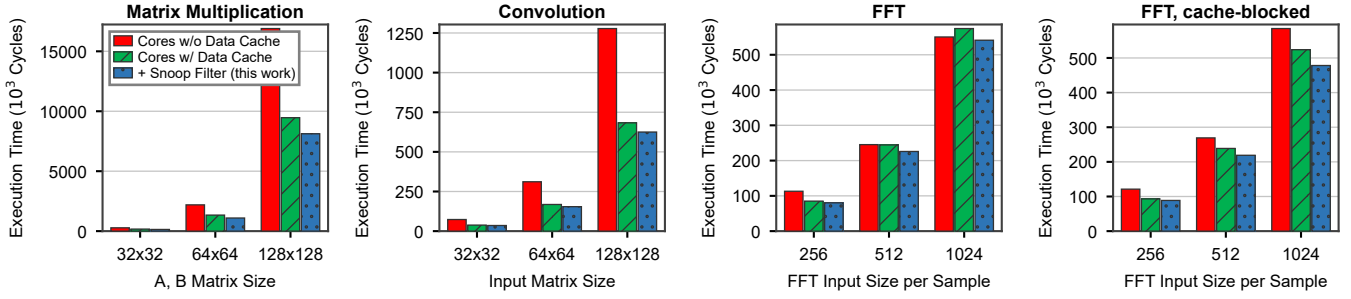


Fig. 2. Execution time of the benchmark kernels over the non-coherent system, the coherent system with the cache extension, and the coherent system with the proposed snoop filter. The fourth panel is the FFT with its stages reordered so that its working set fits into the cache.

evaluation. The reduction does not decrease with the problem size because the data that has to stay resident is not the whole data set but the reuse footprint of the kernel, i.e. the bytes a core revisits before it moves on: one row of the second matrix for matmul, $4N$ bytes for an $N \times N$ matrix of 32-bit words, and the three-row sliding window of the input for conv2d, $12N$ bytes. At the largest problem size these are 512 B and 1536 B, both of which fit into the 2 KiB DCU.

For FFT this no longer holds. The original evaluation reports a uniform reduction of approximately 23 %. We match it at $N=256$ points, but the reduction disappears at $N=512$ and turns negative at $N=1024$, where the cache makes the kernel slower than having no cache. The reuse footprint explains this: the FFT revisits all N complex points in every stage, i.e. $8N$ bytes, which is 2 KiB at $N=256$ and 8 KiB at $N=1024$. Changing one parameter at a time at $N=512$ confirms that the cause is the capacity: doubling the associativity at 2 KiB recovers 3.8 % of execution time, whereas quadrupling the capacity recovers 24.5 %. With an 8 KiB DCU, FFT reproduces the original result at every size (24.8, 24.8 and 24.5 %); with the 2 KiB stated in the original text it does not, so the reported result implies a larger cache or a smaller working set than described.

The collapse is a property of the order in which the kernel accesses its data, and the fourth panel of Fig. 2 removes it in software. A radix-2 butterfly in stage s pairs elements 2^s positions apart; once 2^s is a multiple of a block size B , the remaining stages decompose into B independent transforms over strided columns. Running the early stages block by block and the late stages column by column keeps the working set inside the DCU with every butterfly computed on the same operands, so the output is bit-identical. At $N=1024$ the reduction achieved by the cache extension improves from -4.3% to $+10.4\%$, and to 18.2% with the filter; at $N=256$ the additional loop structure costs about 6 %. The filter removes coherence traffic and not cache misses, so its contribution stays at seven to eight percentage points in both FFT panels.

Relative to the coherent system without the filter, the filter improves the execution time by $1.222\times$ on matmul, $1.123\times$ on memcopy, $1.089\times$ on conv2d and $1.061\times$ on FFT, a geometric mean of $1.122\times$. We also compare against an oracle, a bus that

suppresses every INV REQ broadcast unconditionally, which is sound to simulate only because none of the broadcasts are needed on these kernels. The filter reaches the oracle on every kernel, cycle for cycle, as the cost model of Section V-D predicts. The snoop-stall counter confirms the mechanism: on conv2d it falls from 20,848 cycles to 109, and on memcopy from 7,168 to 1; on FFT it falls only from 6,521 to 2,955, because the LR and SC requests used for synchronization still pass through the arbiters.

B. Memory Copy Latency

The most unexpected finding of the reproduction concerns the memcopy kernel. At a memory latency of two cycles, the coherent system with the cache extension is *slower* than the non-coherent system without any cache, at $0.975\times$ the speed of the baseline; with the filter it is $1.095\times$ faster. The slowdown is the cost of announcing private writes. Fig. 3 shows that this holds for every transfer size: for 4 to 32 KiB the cache extension is between 2.5 % and 7.6 % slower than the baseline, and the filter turns this into a gain of 8.2 % to 9.0 %. Raising the memory latency restores the result of the original evaluation, with a latency reduction of 34 % at a memory latency of eight and 56 % at twenty, against the approximately 50 % reported. A streaming kernel reuses no data, so the cache can only hide memory latency; when there is little latency to hide, the coherence overhead is all that remains.

C. Coherence Verification

The four benchmark kernels never exercise the case in which the filter has to let a broadcast through, so they cannot establish that the filter is safe. Our coherence testbench does: it runs a producer/consumer exchange and a randomized four-core stress test over a small shared window, so that half of its INV REQs are useful, and with the filter enabled it passes 3,112 checks. These include a version-monotonicity check, in which every location carries a version number incremented on every write and a core that has observed version k must never afterwards observe a version below k , the symptom of a lost invalidation. The filter suppressed 282 of 921 broadcasts and delivered every one that was needed.



Fig. 3. Memory copy latency for several data transfer sizes at memory latencies of two, eight and twenty cycles. At two the cache extension is slower than the non-coherent system at every size; the $\approx 50\%$ reduction of the original evaluation appears between eight and twenty.

TABLE II
POST-PLACE-AND-ROUTE RESOURCE UTILIZATION OF THE COHERENT MULTI-CORE SYSTEM WITH THE PROPOSED SNOOP FILTER, AMD/XILINX XCZU7EV AT 60 MHz. TOTALS BELOW THE RULE ARE THOSE REPORTED BY VIVADO.

Module	LUTs	FFs	BRAMs	DSPs
Processing element ($\times 4$)	17,833	9,948	32	20
RISC-V core, CV32E40P	16,929	8,404	0	20
ITCM + Instr.-Bridge	252	36	32	0
Data-Bridge	319	32	0	0
AXI4 masters ($\times 3$ per PE)	333	1,476	0	0
Cache extension	6,540	4,628	20	0
Data Cache Unit ($\times 4$)	5,816	3,998	20	0
Snoopy bus	728	630	0	0
snoop filter (this work)	408	512	0	0
AXI4 crossbar	2,125	279	0	0
Shared data memory, 256 KiB	724	150	64	0
Shared instr. memory, 32 KiB	506	86	8	0
Control region	113	259	0	0
Whole device, as reported by the tool	27,845	15,670	124	20
Available	230,400	460,800	312	1,728
Utilisation (%)	12.09	3.40	39.74	1.16

D. Resource Utilization and Timing

Table II shows the post-place-and-route resource utilization of the coherent multi-core system with the filter, obtained with Vivado 2025.1 for an AMD/Xilinx Zynq UltraScale+ XCZU7EV device at 60 MHz, the clock frequency of the original design. The filter costs 1,064 look-up tables (LUTs) and 516 flip-flops over the coherent system without it, less than 4 % of that system, and no BRAM or DSP blocks. Both systems meet all timing constraints, with a worst negative slack of +1.199 ns with the filter and +1.265 ns without, so the filter costs 66 ps on a 16.667 ns clock period. Both were implemented in the same Vivado session from one source tree with one parameter changed, since separate runs of this design vary by up to 1.7 ns in slack. At a target of 75 MHz, an earlier pair of builds had the system without the filter meeting timing and the filtered system missing it by 2.586 ns, on a critical path that is 86 % routing delay from the registered request address of a DCU to the INV REQ arbiter; as this difference is of the order of the run-to-run variation, we evaluate at 60 MHz.

VII. ALTERNATIVE MECHANISMS

Three mechanisms preceded the filter presented above, and each was rejected by a measurement.

Hit-speculative bypass. A READ REQ assumes that it will hit and does not use the bus; only a miss is forwarded. This was useless, because plain READ REQs are never arbitrated anyway, and incorrect, because the check it removed is the single-writer half of the MRSW invariant: a READ REQ that hits while an INV REQ of another core for the same line is pending returns stale data. It nevertheless passed all eleven functional checks of the benchmark harness.

Restructuring the mirror. Two further implementations of the mirror, built to attack the frequency loss of Section VI, did not close timing at 75 MHz either.

Directed invalidation. The mirror knows not only *whether* another DCU holds the line but *which* ones do, so the bus could deliver the INV REQ only to the members of $S(w)$ and wait only for their readiness. On a stencil kernel in which each PE updates a band of rows and exchanges the boundary rows with its neighbours, it delivers 168 INV REQs, all of them useful, but improves execution time by only 0.1 % to 0.3 %: once suppression has removed every WRITE REQ with an empty sharer set, the remaining cost in (2) belongs to writes that some other DCU has to be told about.

VIII. LIMITATIONS

The central measurement of this paper, that none of the invalidation traffic is useful, is a property of the workloads as much as of the design, and we claim it only for the kernels used by the original evaluation. The stencil kernel of Section VII does share data, and the filter still improves its execution time by $1.11\times$ at a memory latency of twenty. All execution times are obtained from RTL simulation and all resource figures from post-place-and-route reports; no bitstream has been run on a board, and power is not reported because the vectorless estimate of Vivado treats this design as idle.

IX. CONCLUSION

We have reproduced a seamless cache extension for RISC-V multi-core systems and shown that, on the benchmark kernels of its original evaluation, between 6 % and 33 % of the time each core spends in its cache goes to invalidation requests that invalidate nothing. An exact snoop filter, a mirror of the tag and valid bits of all DCUs that also covers lines being fetched, suppresses this traffic for 1,064 LUTs and is worth $1.122\times$.

REFERENCES

- [1] A. Kamaleldin, M. Nickel, S. Wu and D. Göhringer, “Seamless Cache Extension for FPGA-based Multi-Core RISC-V SoC,” in *Proc. IEEE 37th Int. System-on-Chip Conf. (SOCC)*, 2024.
- [2] A. Moshovos, G. Memik, B. Falsafi and A. Choudhary, “JETTY: Filtering Snoops for Reduced Energy Consumption in SMP Servers,” in *Proc. 7th Int. Symp. High-Performance Computer Architecture (HPCA)*, 2001.
- [3] J. F. Cantin, M. H. Lipasti and J. E. Smith, “Improving Multiprocessor Performance with Coarse-Grain Coherence Tracking,” in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 246–257.
- [4] A. Moshovos, “RegionScout: Exploiting Coarse Grain Sharing in Snoop-Based Coherence,” in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 234–245.
- [5] W. Terpstra, “TileLink: A Free and Open-Source, High-Performance Scalable Cache-Coherent Fabric Designed for RISC-V,” in *RISC-V Workshop*, 2017.
- [6] J. Balkind *et al.*, “OpenPiton+Ariane: The First Open-Source, SMP Linux-booting RISC-V System Scaling from One to Many Cores,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.
- [7] A. Li *et al.*, “CIFER: A Cache-Coherent 12nm 16mm² SoC with Four 64-Bit RISC-V Application Cores, 18 32-Bit RISC-V Compute Cores, and a 1541 LUT6/mm² Synthesizable eFPGA,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023.